CONFIDENTIAL · CANDIDATE PREP DOSSIER

Senior / Lead Full-Stack .NET & Azure

Elite 14-Day Interview Preparation Roadmap

Target: 8 YoE · Technical Lead / Principal Engineer Bar

Panel Simulation: Enterprise Architect + Engineering Director

Core Stack: C# / ASP.NET Core · React + TypeScript · SQL Server · Azure + AWS

Prepared as a strict hiring-panel coaching artifact

Contents

- 0. How To Use This Dossier
- 1. 14-Day Preparation Roadmap (Sprint Schedule)
- 2. High-Frequency Technical Questions & Senior Answers
- 3. Tech Lead Behavioral & Agile Scenarios (STAR)
- 4. Differentiating "Bonus Points" Playbook
- 5. Rapid-Fire Closing Round & Questions To Ask Them

0. How To Use This Dossier

This is not a glossary. Every answer is written from the vantage point of an engineer who has owned production incidents, made irreversible architectural trade-offs, and led squads through delivery pressure. Use it three ways:

- **Calibrate, don't memorize.** Panels at this level probe for *judgment under constraint*. Rehearse the trade-off reasoning, not the exact wording.
- **Always close the loop.** Every technical answer should end with the trade-off you accepted and how you'd *measure* whether it was right (latency percentiles, error budget, cost, MTTR).
- **Lead with the decision, then justify.** Senior answers are inverted pyramids: conclusion first, then the "because," then the escape hatch ("where this breaks down is...").

> **Signal the panel is buying:** they stop asking "how" and start asking "when would you *not* do this?" That means you've cleared the knowledge bar and they're testing seniority. Welcome those questions — they are where the offer is won.

1. 14-Day Preparation Roadmap (Sprint Schedule)

Treat these two weeks as a sprint with a demo at the end: the "demo" is the live panel. Timebox ruthlessly. Each day has a **build** (hands-on), a **drill** (recall under pressure), and a **reflect** (write the trade-off in one paragraph).

Phase 1 — Days 1–4: Core Backend Depth

Day 1 — .NET Runtime Internals & Async Model

- **Build:** Write a minimal API that exercises `async/await`, `ValueTask`, and `IAsyncEnumerable` streaming. Attach `dotnet-counters` and watch ThreadPool queue length under load.
- **Study:** Task vs. ValueTask, synchronization context (absent in ASP.NET Core), `ConfigureAwait` myths in Core, thread-pool starvation from sync-over-async (`.Result` / `.Wait()`).
- **Drill:** Explain the lifecycle of one HTTP request from Kestrel socket accept to response flush.
- **Reflect:** When is `ValueTask` a *net negative*?

Day 2 — Memory, GC & Allocation Discipline

- **Build:** Profile a hot path with `dotnet-trace` + Visual Studio allocation profiler. Convert a string-splitting hot loop to `Span<T>` / `ReadOnlySpan<char>` and measure Gen0 collections before/after.
- **Study:** Generational GC, LOH (>85KB) fragmentation, server vs. workstation GC, `ArrayPool<T>`, `stackalloc`, pinned object heap, struct vs. class heuristics.
- **Drill:** "Our p99 spikes every ~30s." Walk the GC hypothesis.
- **Reflect:** The cost model of `Span<T>` (why it can't live on the heap / be a field in an async method).

Day 3 — EF Core vs. Dapper & Data Access Patterns

- **Build:** Same query in EF Core (tracking vs. `AsNoTracking`, split vs. single query, compiled queries) and Dapper. Capture the generated SQL and compare execution plans.
- **Study:** N+1, cartesian explosion, change tracker overhead, `DbContext` lifetime/pooling, bulk operations, when raw Dapper wins.
- **Drill:** Defend a hybrid repository (EF for writes/aggregates, Dapper for read models).

Day 4 — SQL Server Performance & Concurrency

- **Build:** Reproduce a deadlock with two sessions, capture the deadlock graph via Extended Events, resolve by access ordering + covering index.
- **Study:** Clustered vs. non-clustered, composite key column order, included columns, parameter sniffing, SARGability, `READ COMMITTED SNAPSHOT`, `rowversion` optimistic concurrency.
- **Drill:** Read an execution plan aloud: key lookup, scan vs. seek, spill warnings.

Phase 2 — Days 5–7: Frontend & System Architecture

Day 5 — React Performance & State at Scale

- **Build:** Profile a re-render cascade with React DevTools Profiler; fix with `memo`, stable callbacks, selector-based state, and list virtualization.
- **Study:** Redux Toolkit vs. Context vs. server-state (React Query/RTK Query), referential equality, reconciliation, Suspense/concurrent features.
- **Drill:** "Typing in one input re-renders the whole dashboard." Diagnose in 90 seconds.

Day 6 — API & Front-End Architecture

- **Build:** Design a BFF; compare REST vs. GraphQL for an aggregation screen; sketch a micro-frontend seam (Module Federation) and its failure modes.
- **Study:** token storage (`httpOnly` cookie + CSRF vs. localStorage + XSS), pagination strategies, idempotency keys, WebSockets vs. SSE vs. polling.

Day 7 — Cloud Patterns & Zero Trust

- **Build:** Whiteboard an event-driven, multi-tenant pipeline on Azure (Service Bus + Functions + Key Vault + Front Door) and its AWS twin.
- **Study:** managed identity, Zero Trust (verify explicitly / least privilege / assume breach), OAuth2/OIDC flows, private endpoints, Blue/Green + Canary.
- **Reflect:** Multi-tenant isolation trade-offs: pool vs. silo vs. bridge.

Phase 3 — Days 8–11: Technical Leadership & Delivery

Day 8 — Agile Execution & Capacity Mapping

- Rehearse: velocity vs. commitment, capacity math (PTO/ceremonies/support load), skill-to-story mapping, protecting ~20% for tech debt.
- Draft one STAR story on scope creep (see Section 3).

Day 9 — Stakeholder & Delivery-Risk Management

- Rehearse: escalation framing (impact / options / recommendation / ask), status reporting that shields the team without hiding risk, RAID logs.
- Draft the "critical release block" STAR story.

Day 10 — Code Quality, Architecture & Mentoring

- Rehearse: enforcing Clean Architecture/SOLID without dogma, PR governance, handling the "brilliant jerk," lifting an underperformer with a concrete plan.

Day 11 — CI/CD Governance & Test Strategy

- Rehearse: trunk-based vs. GitFlow, PR gates, DORA metrics, test pyramid ownership, automated rollback policy, environment strategy.

Phase 4 — Days 12–14: AI Tooling, Live Design & Mock Loops

Day 12 — AI-Assisted Development Narrative

- Build a crisp 3-minute story: Cursor Composer multi-file refactor, project `.cursorrules`, AI-generated tests you reviewed, a diff you *rejected* for a security/memory reason.

Day 13 — Live System Design Drill (timed)

- 45-min whiteboard: "Design a multi-tenant transaction processing system, 10k TPS, exactly-once semantics." Record yourself. Grade on requirements framing, trade-offs, failure modes, and capacity math.

Day 14 — Executive Mock Loops & Reset

- Two back-to-back mocks: one technical-deep, one leadership/behavioral. Then stop. Light review of your own STAR cards only. Sleep > cramming.

2. High-Frequency Technical Questions & Senior Answers

2.1 C# / ASP.NET Core Web API

ASYNC LIFECYCLE

Q1. Walk me through what actually happens when an `async` controller action awaits a database call under load. Why does this scale better than one-thread-per-request?

The request arrives on a Kestrel I/O thread and is dispatched to a thread-pool thread to run the pipeline. When my handler hits `await _db.Query(...)`, the method is compiled into a state machine. At the await point, if the operation isn't already complete, the continuation is registered and **the thread is returned to the pool** — it is not blocked waiting on the socket/IO completion port. The OS signals completion via IOCP, the continuation is scheduled back onto a pool thread, and execution resumes after the await.

This scales because threads are expensive (~1MB stack each, plus context-switch and scheduling cost). The classic thread-per-request model caps you at a few hundred concurrent blocked requests before the pool is exhausted. With async, a handful of threads service thousands of in-flight requests whose time is dominated by I/O wait. Crucially, ASP.NET Core has **no synchronization context**, so continuations go straight to the pool — which is also why `ConfigureAwait(false)` is a no-op in app code here (it still matters in shared libraries).

Where it breaks down: async gives you nothing for CPU-bound work — you've just moved the burn to a pool thread. And the number-one way teams destroy this benefit is *sync-over-async* (`.Result`, `.Wait()`, `GetAwaiter().GetResult()`), which blocks a pool thread while async work completes, causing **thread-pool starvation**: the pool injects threads slowly (~1/500ms), latency climbs, and eventually requests queue and time out. I've root-caused exactly this — p99 climbing while CPU sat idle — and the fix was going async end-to-end, not adding threads.

VALUETASK & ALLOCATION

Q2. When do you reach for `ValueTask<T>` over `Task<T>`, and when is it a trap?

`Task<T>` is a heap allocation per call. On a **hot path that usually completes synchronously** — a cache hit, a buffered stream read, a pooled resource — that allocation is pure waste at scale. `ValueTask<T>` is a struct that wraps either an already-known result or an underlying `IValueTaskSource`, so the synchronous-completion case allocates nothing. That's why `IAsyncEnumerable` and high-throughput APIs like pipelines use it.

The traps are real and I treat them as rules: a `ValueTask` must be awaited *exactly once*; you must not await it twice, access `.Result` before completion, or store it and await later. If I need to await something multiple times or hand it around, I call `.AsTask()` once and work with that. The honest trade-off: `ValueTask` optimizes allocations but is harder to consume correctly and can actually be *slower* if the operation is usually asynchronous (extra struct copying with no allocation win). My default is `Task` for public APIs and ordinary code; I reach for `ValueTask` only when a profiler shows the allocation matters on a genuinely hot, synchronously-completing path.

GC / LOH

Q3. A service shows periodic latency spikes and rising memory. How do you pin it on GC, and what levers do you pull?

First I confirm with data, not vibes: `dotnet-counters` for `gc-heap-size`, Gen0/1/2 counts, `% time in GC`, and allocation rate; if spikes correlate with Gen2/LOH collections, that's my smoking gun. Then `dotnet-trace` or the VS allocation profiler to find *who* is allocating.

Common root causes and fixes, in order of frequency:

- **LOH fragmentation** — objects >85KB (big byte[], large strings from serialization) go on the Large Object Heap, which historically wasn't compacted, so you get fragmentation and Gen2 pauses. Fix: pool buffers with `ArrayPool<T>` / `RecyclableMemoryStream`, stream instead of buffering whole payloads, and avoid giant intermediate arrays.
- **Excessive Gen0 churn** — allocation-heavy hot loops (LINQ chains, string concatenation, boxing). Fix: `Span<T>` / `stackalloc` for transient slicing, `StringBuilder`/string interpolation handlers, avoid boxing value types.
- **Wrong GC mode** — a throughput server running Workstation GC. Fix: Server GC + concurrent GC for server workloads; this parallelizes collection across cores.
- **Mid-lifetime leaks** — static event handlers, caches without eviction, captured closures. Fix: weak references, bounded caches, `IMemoryCache` with size limits.

Trade-off I name: Server GC trades a larger memory footprint and more background threads for throughput; on a memory-constrained container that can backfire, so I set `DOTNET_gcServer` deliberately and validate against the container memory limit to avoid OOMKills.

MIDDLEWARE

Q4. Design a custom middleware pipeline for correlation IDs, exception handling, and tenant resolution. What are the ordering pitfalls?

The pipeline is a nested chain of `RequestDelegate`s; **order is behavior**. My canonical order: exception-handling/problem-details first (so it wraps everything), then correlation-ID enrichment (so every downstream log carries it), then HTTPS/forwarded-headers, then auth-n, then auth-z, then tenant resolution (needs the authenticated principal), then endpoint routing/execution. I implement cross-cutting concerns as middleware, but anything that needs route/endpoint metadata goes *after* `UseRouting`, and I keep per-request services injected via method injection in `InvokeAsync` rather than constructor injection (middleware is a singleton — constructor-injecting a scoped service is the classic captive-dependency bug).

Pitfalls I've been burned by: (1) registering exception handling after the component that throws during startup of the response; (2) writing to the response body and then a later middleware trying to set headers — once the response has started, headers are immutable, so I register an `OnStarting` callback instead; (3) capturing a scoped `DbContext` in the singleton middleware. For tenant resolution specifically I resolve into a scoped `ITenantContext` so the rest of the request — including EF query filters — transparently scopes to the tenant.

2.2 SQL Server & Data Engineering

QUERY LATENCY

Q5. A previously-fast query is now slow in production only. Walk your diagnostic path.

I resist rewriting anything until I understand the plan. Path: capture the **actual execution plan** (not estimated) and the query's stats from `sys.dm_exec_query_stats`/Query Store. I'm looking for: seek vs. scan, a **key lookup** that's now expensive at volume, estimated-vs-actual row count skew (stale statistics), spill-to-tempdb warnings (under-sized memory grant), and whether the same SQL has two wildly different plans (**parameter sniffing**).

"Fast before, slow now, same query" overwhelmingly points to one of: **stale statistics** after a large data change (fix: update stats / let auto-update catch up, or Query Store plan forcing), **parameter sniffing** where a plan compiled for an atypical parameter got cached (fix: `OPTIMIZE FOR`, `RECOMPILE` on the outlier, or branch the query), **blocking/locking** from a concurrent workload (check `sys.dm_exec_requests` wait types), or **index fragmentation/missing covering index** as the table grew. I validate the hypothesis against Query Store's regression view, which shows me the exact moment and plan that changed — that's the artifact I bring to the conversation rather than a guess.

DEADLOCKS

Q6. How do you resolve recurring deadlocks under heavy write traffic — without just adding retries?

Retries are a seatbelt, not a fix. First I capture the **deadlock graph** (Extended Events `xml_deadlock_report`) to see the exact resources and the two statements involved. Deadlocks are almost always one of: (1) **inconsistent lock ordering** — transaction A locks table X then Y, transaction B locks Y then X. Fix: enforce a canonical access order across all code paths. (2) **lock escalation / wide scans** taking more locks than needed. Fix: a covering/narrower index so the writer seeks instead of scans, reducing the lock footprint. (3) **long transactions** holding locks across round-trips. Fix: shrink the transaction scope, move non-transactional work out.

Then I add idempotent retry-on-1205 with jittered backoff as defense in depth, and I consider `READ COMMITTED SNAPSHOT` to take readers out of the writers' way (row-versioning in tempdb) — naming the trade-off that RCSI increases tempdb pressure and changes read semantics, so I validate tempdb sizing first. The deliverable is the deadlock graph plus the specific index/ordering change, not "we turned on retries."

INDEXING

Q7. Clustered vs. non-clustered, and how do you choose composite-index column order?

The **clustered index is the physical table order** — there's exactly one, and it should usually be narrow, static, ever-increasing (to avoid page splits) and the column you most often range-scan/join on. A GUID clustered key is a classic anti-pattern because random inserts cause fragmentation and page splits; I'd use a sequential key (identity or sequential GUID) and keep the random GUID as a non-clustered unique key if needed.

Non-clustered indexes are separate B-trees pointing back to the clustered key. For composite order I use the **Equality, Range, then Include** rule: leading columns are the ones used with equality predicates (highest selectivity / most common filter first), then the range column, and I push columns that are only *selected* (not filtered) into `INCLUDE` to make the index **covering** — eliminating key lookups. Column order matters because a composite index on (A,B) can seek on A or A+B but not B alone. I justify every index against write cost: each index is maintained on every insert/update/delete, so I audit for unused and duplicate indexes (`sys.dm_db_index_usage_stats`) rather than carpet-bombing the table with indexes to fix one slow read.

CONCURRENCY

Q8. Optimistic vs. pessimistic concurrency under heavy traffic — which and why?

Default to **optimistic** for high-contention, read-heavy web traffic: a `rowversion` / `timestamp` column, and on update I include the original version in the WHERE clause; zero rows affected means someone beat me, and I surface a conflict (merge, retry, or user-resolve). This holds no locks across the user's think-time, so it scales. EF Core does this natively with `[ConcurrencyCheck]` / `IsConcurrencyToken` throwing `DbUpdateConcurrencyException`.

Pessimistic (explicit locks, `UPDLOCK` / `HOLDLOCK`, or app-level locks) I reserve for short, high-value critical sections where a conflict is expensive to reconcile — e.g., decrementing scarce inventory or allocating a unique seat/number — where I'd rather serialize briefly than let two winners proceed. The trade-off is throughput and deadlock risk, so I keep the locked region tiny. For money movement specifically I lean on the database's atomicity plus an **idempotency key** so a client retry can't double-apply — concurrency control and idempotency are different problems and I address both explicitly.

2.3 React.js & Full-Stack Integration

STATE AT SCALE

Q9. How do you structure state in a large React app so it stays performant and maintainable?

My first move is to **separate server state from client state** — the single biggest architectural lever. Server/cache state (anything fetched from an API) belongs in React Query or RTK Query: it gives me caching, dedup, background refetch, and stale-while-revalidate for free, and it deletes the mountain of Redux boilerplate teams write to manually track loading/error/data. Genuine *client* state — auth/session, feature flags, theme, cross-cutting UI — goes in Redux Toolkit (or a lightweight store like Zustand) with selectors. Ephemeral, local UI state stays in component `useState`.

I use **Context sparingly**: Context is a dependency-injection mechanism, not a state manager — every consumer re-renders on any value change, so I only put low-frequency, broadly-needed values there and I split providers so a theme change doesn't re-render the data grid. At scale the discipline that pays off is colocation (state lives as close to where it's used as possible) and selector-based reads so components subscribe only to the slice they need.

RENDERING

Q10. A dashboard re-renders excessively. How do you find and fix the cascade?

Diagnose with the **React DevTools Profiler** — "highlight updates" and the flamegraph show me exactly which components render and *why* (props/state/parent). The usual culprits: a new object/array/function literal created in render and passed as a prop (breaks referential equality every render), lifting state too high so a keystroke re-renders siblings, or an unstable Context value.

Fixes, applied surgically (not sprinkled everywhere): `React.memo` on pure leaf components that get the same props often; `useCallback` / `useMemo` to stabilize the *references* I pass down (only where a memoized child or an effect dependency actually needs it); move state down or use a store with selectors so the keystroke only re-renders the input; and **virtualize long lists** (`react-window`) because the cheapest render is the one that never mounts. I explicitly avoid cargo-culting `useMemo` everywhere — memoization has its own cost and masks the real problem, which is usually bad data flow. I verify the fix by re-profiling and comparing commit counts, not by eyeballing.

TOKEN SECURITY

Q11. Where do you store the auth token in a SPA — localStorage or cookies? Defend it.

I store tokens in a `Secure` , `HttpOnly` , `SameSite` **cookie**, not localStorage. The reasoning is a threat-model trade-off: localStorage is readable by any JavaScript on the page, so a single successful **XSS** payload (a compromised npm dependency, a reflected input) exfiltrates the token instantly — and XSS is the more common, higher-impact web vulnerability. An `HttpOnly` cookie is invisible to JS, so XSS can't read it.

The cost of cookies is **CSRF** exposure, which I mitigate explicitly: `SameSite=Lax/Strict` plus anti-forgery tokens (double-submit) for state-changing requests, and tight CORS. The pattern I actually ship is a **BFF** (backend-for-frontend): the SPA talks to its own backend over an `HttpOnly` session cookie, and the backend holds the real OAuth access/refresh tokens server-side, exchanging them when calling downstream APIs. The browser never sees a bearer token at all. I pair this with short-lived access tokens + refresh rotation, a strict CSP to reduce XSS surface in the first place, and I treat "no XSS" as the real goal — cookie storage just limits the blast radius when one slips through.

REALTIME

Q12. The product needs "live" data. How do you choose between polling, SSE, and WebSockets, and keep it resilient?

I choose by data shape and direction, not by what's trendy. **Polling** (ideally long-poll or interval with conditional `ETag`/304) is fine for low-frequency, tolerant updates and is the most operationally boring — which is a virtue. **SSE** for one-way server-to-client streams (notifications, progress, dashboards) — it's just HTTP, auto-reconnects, and sails through proxies. **WebSockets** (SignalR in .NET) when I need low-latency *bidirectional* comms — chat, collaborative editing, trading.

Resilience is where senior shows: for WebSockets I plan for the connection *failing*, not succeeding — exponential backoff with jitter on reconnect, a heartbeat/ping to detect half-open sockets, message sequence numbers so I can resync missed events on reconnect, and a fallback to SSE/long-poll (SignalR negotiates this automatically). At scale I name the stateful-scale-out problem: sticky sessions or a **backplane** (Redis/Azure SignalR Service) so a message published on one instance reaches clients connected to another. For polling I always set conditional requests and backoff so a degraded backend doesn't get hammered by thousands of clients retrying in lockstep.

2.4 Cloud Architecture (Azure & AWS)

SYSTEM DESIGN · FLAGSHIP

Q13. Design an event-driven, multi-tenant transaction processing system. Target ~10k TPS, exactly-once effect, zero-downtime deploys. Use Azure, and tell me the AWS equivalent.

1. Clarify first (always). I pin down: TPS and burst shape, payload size, latency SLO, ordering requirement (per-account vs. global), tenant count and isolation/compliance needs, and what "exactly-once" means to the business (I'll deliver *effectively-once* via idempotency, since true exactly-once delivery is a distributed-systems myth).

2. Ingest & edge. Azure Front Door (global LB + WAF + TLS + caching) in front of **API Management** for throttling, per-tenant rate limits, and JWT validation at the edge. The intake API is thin: validate, authenticate the tenant, assign an **idempotency key**, and publish — it does not do the work inline.

3. Messaging backbone. Azure Service Bus for the command/transaction path: it gives me FIFO via *sessions* (session id = account id for per-account ordering), dead-letter queues, duplicate detection, and competing-consumer scale-out. For the high-throughput *event* fan-out (audit, analytics, projections) I'd use **Event Hubs** (partitioned log, Kafka-semantics). Service Bus for commands I can't lose and must order; Event Hubs for high-volume telemetry/event streams.

4. Compute. Azure Functions (or Container Apps/AKS if I need more control) as competing consumers off the queue, scaling on queue depth (KEDA). Processing is **idempotent**: before applying a transaction I check the idempotency key against a dedup store (Cosmos DB/SQL), so a redelivery (Service Bus is at-least-once) is a no-op. This is how I get effectively-once.

5. Persistence & multi-tenancy. I present the **tenancy spectrum** and justify a choice: *silo* (DB per tenant — strongest isolation/compliance, costlier, more ops), *pool* (shared schema + tenant-id + row-level security — cheapest, best density, noisy-neighbor risk), *bridge* (shared DB, schema per tenant). For a transaction system I typically go pool with row-level security and a per-tenant partition key for the long tail, and silo for enterprise tenants with compliance mandates — a pragmatic hybrid. Writes to Azure SQL (ACID, the system of record) + an event/outbox for projections.

6. Consistency. Transactional outbox pattern so the DB write and the "event published" fact commit atomically; a relay publishes from the outbox. For multi-step cross-service flows, a **saga** with compensating actions — not a distributed 2PC.

7. Secrets & identity. Key Vault for secrets/keys, accessed via **managed identity** — no secrets in config. Zero Trust throughout (see Q14).

8. Zero-downtime deploys. Blue/Green via deployment slots (warm the green slot, run smoke tests, swap) for the APIs, and **Canary** (Front Door weighted routing / 5%→25%→100%) for risky changes, gated on live error-rate/latency SLIs with automated rollback. Consumers are versioned and schema changes are **expand/contract** so old and new run concurrently during the swap.

9. Observability & resilience. Application Insights + OpenTelemetry for distributed tracing across the async hops (the correlation id rides the message), DLQ alerting, and Polly for retries/circuit-breakers on downstream calls. I define SLOs and an error budget and wire alerts to budget burn, not raw error counts.

Concern	Azure	AWS equivalent
Edge / CDN / WAF	Front Door	CloudFront + WAF
API gateway / throttling	API Management	API Gateway
Ordered/reliable commands	Service Bus (sessions)	SQS FIFO
High-volume event stream	Event Hubs	Kinesis / MSK
Serverless compute	Azure Functions	Lambda
Containers	Container Apps / AKS	ECS Fargate / EKS
Relational store	Azure SQL	RDS / Aurora
Secrets	Key Vault + Managed Identity	Secrets Manager + IAM Roles
Observability	App Insights / Monitor	CloudWatch / X-Ray

The one-liner I close with: "At-least-once delivery + idempotent consumers + transactional outbox is how I get exactly-once *effect* without pretending exactly-once *delivery* exists."

ZERO TRUST

Q14. What does Zero Trust actually mean in the system above — concretely, not as a buzzword?

Three principles, each with a concrete control: **Verify explicitly** — every service-to-service call is authenticated with **managed identity** and Entra ID, not a shared secret or network trust; JWTs are validated (signature, issuer, audience, expiry) at API Management *and* at the service. **Least privilege** — each identity gets scoped RBAC to exactly the resources it needs (this function can read *this* Key Vault secret and write *this* queue, nothing else), and data access is row-level-security scoped to the tenant. **Assume breach** — network segmentation with private endpoints so data stores aren't publicly reachable, secrets in Key Vault with rotation, encryption in transit and at rest, and full audit logging so I can reconstruct a compromise. The mental shift is that the network perimeter is *not* a trust boundary — identity is.

3. Tech Lead Behavioral & Agile Scenarios (STAR)

Deliver these conversationally, ~2 minutes each, and always land the **result with a metric** plus a one-line "what I'd do differently." Panels reward self-awareness as much as the win.

3.1 Resource Mapping & Scope Creep

Situation: Mid-sized squad (6 engineers), a committed quarterly roadmap, and a payments-rewrite that had accumulated serious tech debt. Two weeks into the quarter a major client escalated through our VP demanding a net-new reporting feature "by month end."

Task: As tech lead I owned both the delivery commitment and the health of the codebase. I had to satisfy a real revenue-impacting stakeholder *without* torching the team's capacity or letting the debt compound into an outage.

Action: I refused to say a blind "yes" or "no." I ran a quick capacity map: real velocity minus support rotation and PTO, then decomposed the request with two seniors into a thin vertical slice (the 20% of the feature that delivered 80% of the client's actual need). I laid out three options for the VP with explicit trade-offs: (A) full feature, slips the payments milestone 3 weeks; (B) the thin slice in two sprints, payments milestone intact; (C) staff a contractor, highest cost. I was transparent that the payments debt was a reliability risk, not a vanity refactor, and quantified it (recent incident rate). We chose B. I protected ~20% of sprint capacity for the debt and tracked both streams on one board.

Result: Delivered the thin slice in 11 days; the client signed the renewal; the payments milestone held; and the carved-out debt work cut the module's incident rate by roughly 60% that quarter. *What I'd do differently:* I'd have socialized a lightweight intake/change process earlier so the next escalation didn't need a fire drill to triage.

3.2 Delivery Bottleneck & Status Reporting

Situation: Two days before a contractually-committed release, integration testing surfaced a data-corruption edge case in a downstream integration we didn't own. Upper management was already briefing the client on the launch date.

Task: Decide whether to ship, hold, or partially release — and communicate it up the chain honestly while keeping my engineers focused on the fix rather than the politics.

Action: I immediately separated signal from noise: reproduced the bug, scoped blast radius (which tenants, what data, reversibility), and confirmed it was a correctness issue — a hard no-ship for a payments path. I shielded the team by taking the status reporting entirely onto myself and giving them a single focused objective. To management I didn't bury the lede: I led with impact ("risk of silent data corruption for ~3% of tenants"), gave options (full hold vs. feature-flagged release to unaffected tenants), a recommendation (flagged partial release), and a concrete ask (one more day + a go/no-go at 9am). I set up a short, scheduled update cadence so leadership felt informed and stopped pinging the engineers directly.

Result: We shipped behind a feature flag to unaffected tenants on the committed date, held the risky path one extra day, and fixed it with the partner. Zero corruption incidents, the client relationship stayed intact, and leadership explicitly called out the transparency. *Lesson reinforced:* executives can handle bad news — what erodes trust is *surprise*, so I report risk early with options attached.

3.3 Architecture Conflict, Code Quality & Mentoring

Situation: A strong but opinionated senior kept merging large PRs that bypassed our Clean Architecture boundaries (business logic leaking into controllers, direct data access from the API layer), while a junior on the team was quietly stalling — PRs sitting for days, visibly under-confident.

Task: Protect architectural integrity and consistency without turning it into a personality war, and simultaneously unblock and grow the struggling engineer.

Action: On the architecture conflict, I moved the debate out of PR comments (where it was getting personal) into a 30-minute design session and reframed it around *shared principles we'd agreed on* — testability and change isolation — not my preference vs. his. We codified the boundaries as an **ADR** and, crucially, enforced them in CI with an architecture test (NetArchTest) and PR templates, so the rule was the system's, not mine. That depersonalized it and he actually became an advocate. For the junior, I paired him with a senior, broke his work into smaller reviewable PRs, gave him a low-risk but visible component to own, and ran weekly 1:1s focused on confidence and specific feedback rather than volume.

Result: Architecture violations dropped to near-zero because CI caught them before review, and review time fell since debates were settled by policy. The junior shipped his owned component within six weeks and was independently taking stories by the next quarter. *What I'd do differently:* establish the ADR + architecture tests at project kickoff — enforcing conventions after they've drifted is far more expensive than encoding them up front.

4. Differentiating "Bonus Points" Playbook

4.1 Cursor AI IDE in Production — Force Multiplier, Not Crutch

The framing that lands with a senior panel: **"AI accelerates the parts I already know how to verify; it never makes the engineering decision."** You want to signal leverage *and* judgment. Concrete talking points:

- **Composer multi-file refactors:** "When I renamed a bounded context and needed to propagate an interface change across ~30 files — DTOs, handlers, tests — I drove it with Composer and reviewed the diff as a single reviewable unit. It compressed an afternoon of mechanical edits into ~20 minutes, and I still read every hunk before committing." Emphasize that the human owns the *intent* and the review.
- **Project-specific `.cursorrules`:** "I encode our conventions — Clean Architecture boundaries, async-all-the-way, no `.Result`, our logging and error-handling patterns, naming — into `.cursorrules` so AI output is born compliant. It's effectively executable onboarding docs; it also keeps generated code consistent across the squad instead of each dev getting a different style."
- **AI-assisted unit tests:** "I use it to generate the *boring* coverage — the arrange/act boilerplate and the obvious edge cases — which frees me to hand-write the tricky concurrency and boundary tests that actually matter. I treat generated tests with suspicion: a test that passes but asserts the wrong thing is worse than no test, so I review assertions, not just green checkmarks."
- **Auditing AI diffs for memory/security gaps:** This is the maturity signal. "I actively look for the failure modes AI is prone to: a disposed resource not wrapped in `using`, a captured closure causing a leak, sync-over-async sneaking in, SQL built via string interpolation instead of parameters, secrets hardcoded, or over-broad exception swallowing. AI is confident and fluent, which is exactly why it needs a skeptical senior reading the diff."

> **Trap to avoid:** never imply you ship AI output unreviewed or that you "couldn't do it without AI." The hireable posture is: "It's a power tool. I'm accountable for every line that merges under my name, AI-authored or not."

4.2 Testing Strategy & SRE

The Test Pyramid I actually enforce

- **Unit (broad base):** xUnit/NUnit + Moq/NSubstitute. Fast, deterministic, test behavior not implementation. These run on every push and gate the PR.
- **Integration (the high-value middle): Testcontainers** to spin up a real SQL Server / Service Bus emulator / Redis in a container per test run. This is the tier that catches what unit tests mock away — actual SQL, real transactions, migration correctness — without the flakiness of shared environments. I push hard for this tier because mocked integration tests give false confidence.
- **E2E (thin top): Playwright** for a small number of critical user journeys (login, checkout, the money path). Kept deliberately thin because E2E is slow and flaky at volume; I use it to protect business-critical flows, not to chase coverage.

SRE & Observability

- **SLIs/SLOs & error budgets:** I define SLOs (e.g., 99.9% of requests <300ms) and manage to an **error budget**. The budget is a shared language with the business: budget healthy → ship fast; budget burning → freeze features and spend on reliability. It turns "how stable is stable enough" from an argument into a policy.
- **Observability via OpenTelemetry / Application Insights:** distributed tracing with a correlation id that survives async/queue hops, structured logs, and the RED/USE metrics. The goal is reducing **MTTR** — when paged, I want to go from alert to root cause in minutes via a trace, not grep across ten services.
- **CI/CD governance & automated rollback:** trunk-based with PR gates (build, tests, SAST, coverage threshold), progressive delivery (canary/blue-green), and **automated rollback** wired to SLI breach — if error rate or latency crosses threshold post-deploy, the pipeline rolls back without a human in the loop. I track **DORA metrics** (deployment frequency, lead time, change-failure rate, MTTR) as the health dashboard for the delivery system itself.

5. Rapid-Fire Closing Round & Questions To Ask Them

Rapid-fire self-check (answer each in <30s)

Prompt	One-line senior answer
`IEnumerable` vs `IQueryable` in a repo signature?	Return `IEnumerable` from the boundary; `IQueryable` leaks the provider and risks client-eval / open-ended queries downstream.
Why not `async void`?	Exceptions can't be awaited/caught — they crash the process. Only valid for event handlers.
`record` vs `class`?	Records for immutable value-semantics DTOs/messages; classes for entities with identity & behavior.
CTE vs temp table vs table variable?	CTE for readability (not a perf tool); temp table when the optimizer needs stats on a large intermediate; table variable for tiny sets.
When does a microservice split hurt?	When services share a database or deploy together — you've bought distributed-systems cost with none of the autonomy benefit.
Idempotency key placement?	At the command boundary, checked before side effects, stored with the result so retries return the original outcome.

Smart questions to ask the panel (signals seniority)

- "What does your incident/on-call culture look like — blameless postmortems, and who owns MTTR?"
- "How is the boundary between this lead role's technical authority and the EM's people authority drawn?"
- "Where's your biggest architectural regret right now, and is there appetite to pay it down?"
- "How do you decide what's a platform/shared concern vs. owned by a product squad?"
- "What's the current change-failure rate and lead time — and what would you want them to be in a year?"

Prepared as a strict hiring-panel coaching artifact. Calibrate the specifics (team sizes, metrics, incident numbers) to your own real history — the frameworks are the scaffolding; your authentic stories are what win the offer.